

Construire et manipuler un environnement IA avec RAG

Projet « Understanding Retrieval-Augmented Generation » — Base de connaissance vectorielle locale (PostgreSQL/pgvector) + LLM local (Ollama).

Docker

PostgreSQL 16 + pgvector

Ollama

gemma2:2b

nomic-embed-text

Python 3.14

1. Objectif

Le but du projet est de construire, de bout en bout, un environnement d'intelligence artificielle exploitant le principe du **RAG (Retrieval-Augmented Generation)** : enrichir les réponses d'un grand modèle de langage (LLM) avec une base de connaissance personnalisée, afin qu'il réponde sur des données qu'il n'a jamais vues lors de son entraînement. La démonstration s'appuie sur une entreprise fictive, **HelioniX SAS** (onduleurs photovoltaïques), dont les documents internes servent de base de connaissance.

2. Environnement construit

L'environnement repose sur la **virtualisation applicative (conteneurs Docker)** plutôt que sur une machine virtuelle complète : chaque outil tourne dans un conteneur isolé, orchestré par `docker compose`. Ce choix réduit l'empreinte et rend l'environnement reproductible en une commande.

Couche	Élément	Détail
Machine hôte	Windows 11 Pro (x86-64)	Exécute Docker Desktop (backend WSL 2) et l'interpréteur Python
Virtualisation	Docker Desktop 28.x	Conteneurisation, orchestration via <code>docker-compose.yml</code>
Base vectorielle	conteneur <code>rag-db</code>	image <code>pgvector/pgvector:pg16</code> — PostgreSQL 16 + extension <code>vector</code> , exposé sur <code>localhost:5433</code>
Moteur LLM	conteneur <code>rag-ollama</code>	image <code>ollama/ollama</code> — sert l'embedding et la génération, exposé sur <code>localhost:11434</code>
Modèle d'embedding	<code>nomic-embed-text</code>	Transforme un texte en vecteur de 768 dimensions
Modèle de génération	<code>gemma2:2b</code> (Google)	LLM léger (~1,6 Go) adapté à une exécution CPU locale
Scripts	Python 3.14 (venv)	<code>psycopg2-binary</code> (PostgreSQL) + <code>requests</code> (API Ollama)

Schéma d'architecture

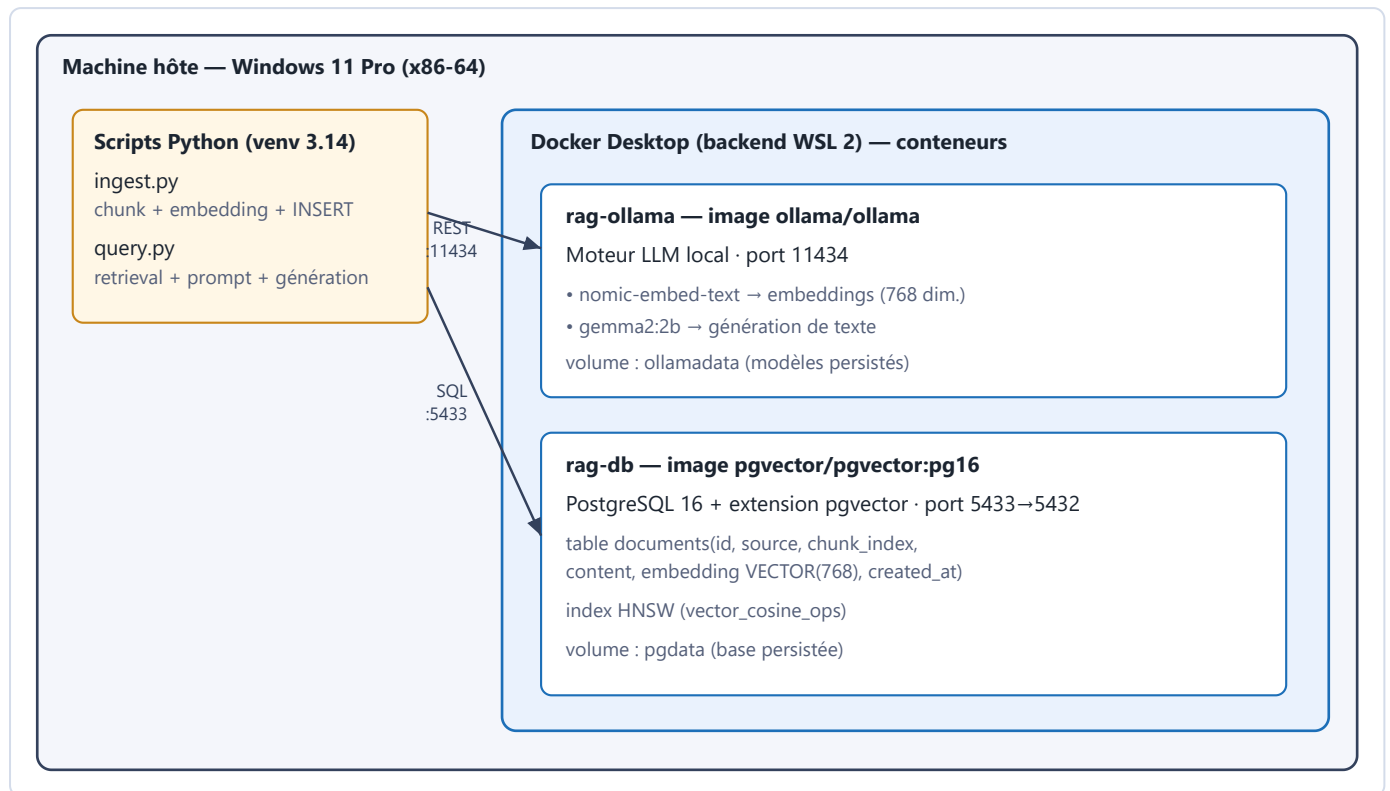


Figure 1 — Architecture de l'environnement : scripts Python sur l'hôte, conteneurs Ollama et PostgreSQL/pgvector orchestrés par Docker. Les scripts dialoguent avec Ollama en REST (port 11434) et avec PostgreSQL en SQL (port 5433).

3. Workflow des traitements

Le pipeline se décompose en deux phases : une phase d'**ingestion** (hors ligne, exécutée une fois pour peupler la base) et une phase d'**interrogation** (rejouée à chaque question de l'utilisateur).

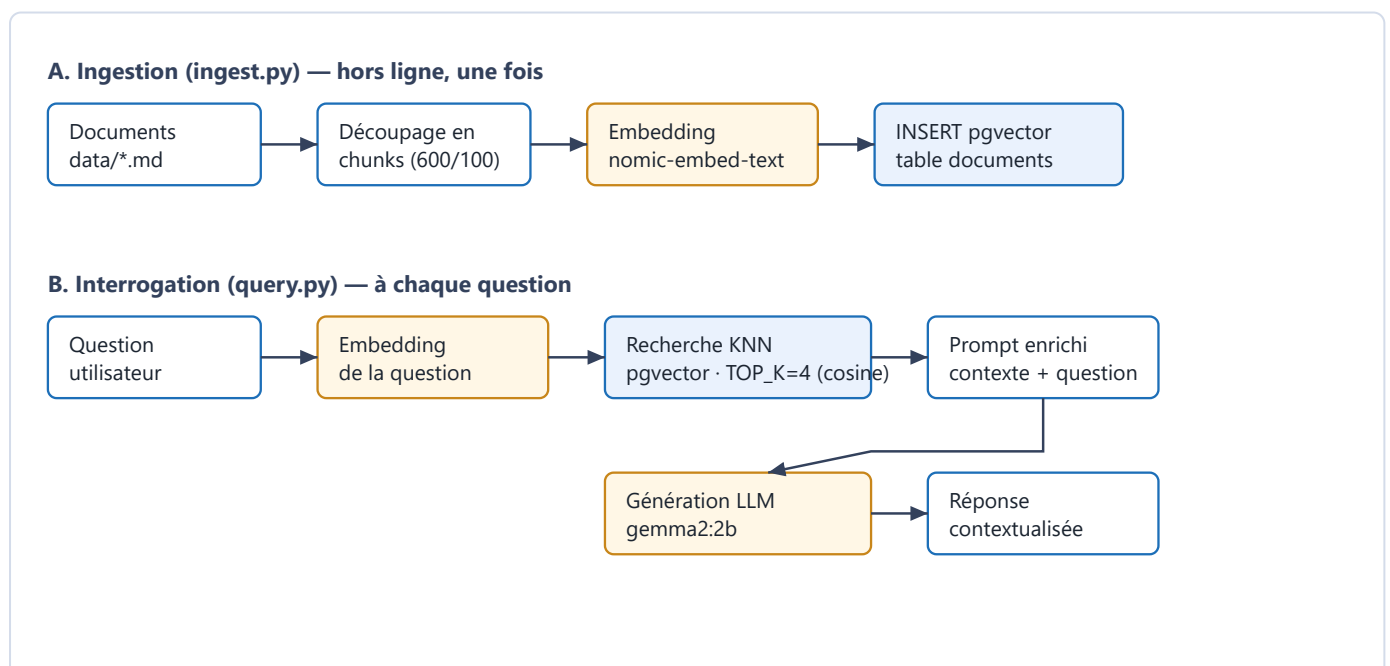


Figure 2 — Workflow : (A) ingestion des documents, (B) interrogation contextualisée du LLM par recherche de similarité vectorielle.

4. Procédure d'installation

L'ensemble de l'environnement se déploie depuis le dossier du projet.

4.1 — Démarrage des conteneurs (PostgreSQL/pgvector + Ollama)

```
# Lancement de la base vectorielle et du moteur LLM
docker compose up -d
docker compose ps          # vérifier que rag-db est "healthy" et rag-ollama "up"
```

Au premier démarrage, le script `db/init.sql` est joué automatiquement : il active l'extension `vector`, crée la table `documents` et son index HNSW.

4.2 — Téléchargement des modèles dans Ollama

```
# Modèle d'embedding (768 dimensions) et modèle de génération
docker exec rag-ollama ollama pull nomic-embed-text
docker exec rag-ollama ollama pull gemma2:2b
docker exec rag-ollama ollama list      # vérifier les modèles disponibles
```

4.3 — Environnement Python et dépendances

```
python -m venv .venv
.venv\Scripts\activate          # (Windows)
pip install -r projet_cyber_rag/requirements.txt
# requirements : psycpg2-binary==2.9.12, requests==2.32.3
```

4.4 — Ingestion puis interrogation

```
# Peupler la base de connaissance (découpage + embeddings + insertion)
python scripts/ingest.py --reset

# Poser une question (avec contextualisation RAG)
python scripts/query.py "Quelle est la garantie de l'onduleur HX-Solar 5000 ?"
# Comparaison : interroger le LLM seul, sans la base
python scripts/query.py --no-rag "..."

# Batterie de démonstration complète (génère le tableau prompt/réponse)
python scripts/demo.py
```

5. Scripts développés

5.1 — Ingestion : `ingest.py`

Lit les documents, les découpe en fragments (*chunks*) de ~600 caractères avec 100 de recouvrement, calcule l'embedding de chaque fragment via Ollama et l'insère dans pgvector.

```
def chunk_text(text, size, overlap):
    text = text.strip()
    if not text:
        return []
    chunks, start, n = [], 0, len(text)
    while start < n:
        end = min(start + size, n)
        if end < n:                                # coupe de préférence sur un saut de ligne
            window = text.rfind("\n", start + size // 2, end)
            if window > start:
                end = window
        chunk = text[start:end].strip()
        if chunk:
            chunks.append(chunk)
        if end ≥ n:                                # tout le texte consommé → fin
            break
        start = max(end - overlap, start + 1)      # progression stricte
    return chunks

def embed(text):
    # embedding via Ollama (nomic-embed-text)
    r = requests.post(f"{OLLAMA_URL}/api/embeddings",
                      json={"model": EMBED_MODEL, "prompt": text}, timeout=120)
    r.raise_for_status()
    return r.json()["embedding"]

# ... pour chaque chunk : INSERT documents(source, chunk_index, content, embedding)
```

5.2 — Interrogation : `query.py`

Vectorise la question, récupère les `TOP_K=4` fragments les plus proches (distance cosinus via l'opérateur `↔` de pgvector), construit un prompt contraint au contexte puis génère la réponse avec `gemma2:2b`.

[illegible]

6. Démonstration — couples prompt / réponse

Le tableau ci-dessous met en évidence la **bonne prise en compte de la base de connaissance** : les réponses « Avec RAG » citent des faits précis issus des documents HelioniX (garantie, codes d'erreur, politique RH...). Pour 3 questions, la réponse « Sans RAG » (LLM seul, sans la base) est fournie en regard : le modèle est alors incapable de répondre correctement, ou refuse, ce qui démontre l'apport du RAG. La dernière question porte volontairement sur un sujet **hors base** : le système répond qu'il ne dispose pas de l'information, prouvant qu'il ne « brode » pas.

#	Prompt	Chunks récupérés	Réponse
1	Quelle est la garantie constructeur de l'onduleur HX-Solar 5000 ?	entreprise_helionix.md, politique_rh_helionix.md, produit_hx_solar_5000.md score 0.812	Avec RAG : 12 ans Sans RAG : La garantie constructeur de l'onduleur HX-Solar 5000 est de **10 ans** sur le matériel et **5 ans** sur les composants électroniques.
2	Que signifie le code d'erreur E-204 sur le HX-Solar 5000 ?	entreprise_helionix.md, politique_rh_helionix.md, produit_hx_solar_5000.md score 0.787	Avec RAG : Le code d'erreur E-204 signale une surchauffe du transformateur : il faut vérifier la ventilation et contacter le SAV HelioniX si le défaut persiste. Sans RAG : Le code d'erreur E-204 sur le HX-Solar 5000 indique un **défaut de communication avec la centrale solaire** .
3	Combien de jours de télétravail par semaine sont autorisés chez HelioniX, et quel jour est obligatoirement travaillé au bureau ?	entreprise_helionix.md, politique_rh_helionix.md, produit_hx_solar_5000.md score 0.803	Avec RAG : HelioniX autorise jusqu'à 3 jours de télétravail par semaine pour les postes éligibles. Le mardi est un jour de présence obligatoire au bureau pour toutes les équipes (réunion hebdomadaire « SunSync »). —
4	Quel est le rendement européen du HX-Solar 5000 et sur quel port TCP fonctionne le protocole SunLink v3 ?	politique_rh_helionix.md, produit_hx_solar_5000.md score 0.786	Avec RAG : Le rendement européen du HX-Solar 5000 est de 97,8 %. Le protocole SunLink v3 fonctionne sur le port TCP 8770. —
5	Qui est la présidente de HelioniX et quel était le chiffre	entreprise_helionix.md, politique_rh_helionix.md,	Avec RAG : Amélie Rouanet est la présidente de HelioniX.

	d'affaires 2024 ?	produit_hx_solar_5000.md score 0.757	Le chiffre d'affaires 2024 était de 38,6 millions d'euros. —
6	Quelle est la valeur faciale du ticket restaurant et la prise en charge employeur chez HelioniX ?	entreprise_helionix.md, politique_rh_helionix.md, produit_hx_solar_5000.md score 0.778	Avec RAG : La valeur faciale du ticket restaurant est de 11 € et la prise en charge employeur est de 6,60 €. —
7	Quelle est la capitale de l'Australie ?	entreprise_helionix.md, politique_rh_helionix.md, produit_hx_solar_5000.md score 0.597	Avec RAG : Je ne dispose pas de cette information dans ma base de connaissance. Sans RAG : Canberra AU

7. Conclusion

L'environnement répond aux trois objectifs du projet : **(1)** construction d'une plateforme IA locale conteneurisée (Ollama + PostgreSQL/pgvector), **(2)** ingestion d'un jeu de documents dans une base de connaissance vectorielle, et **(3)** interrogation d'un LLM enrichi par cette base. La comparaison « avec / sans RAG » confirme que la base de connaissance est effectivement mobilisée et qu'elle améliore nettement la pertinence des réponses sur des données métier privées.